Serial Communication Protocols

By,
Raghudathesh G P

Dr. Raghudathesh G P

Serial Communication Protocols — Comparison

How to choose between them, and what each one is actually for

On-board, chip to chip

Centimetres of copper on one PCB. Short, fast, cheap. No transceivers, no isolation, no error checking beyond what you add yourself.
I2C, SPI, I2S, 1-Wire, QSPI, SDIO

Board to board, industrial

Metres to hundreds of metres, often through electrical noise. Differential signalling and transceiver chips appear here.
UART, RS-232, RS-422, RS-485, CAN, LIN

Host and network

Connecting to a PC or a network. Complex protocol stacks, hot-plug, enumeration, addressing.
USB, Ethernet

The three questions that decide the choice

How far?	Centimetres on a board, or metres through a factory? Distance is what forces differential signalling.
How fast?	A 10 Hz temperature sensor and a colour display have nothing in common.
How many devices?	One partner, or a dozen sharing the same pair of wires? This decides whether you need addressing.

Answer those three and the protocol usually picks itself.

Dr. Raghudathesh G P

On-Board Protocols — Chip to Chip

Short runs on one PCB. No transceivers, no isolation, minimal error checking.

Protocol	Type	Wires	Max distance	Max speed	Topology	Typical usage
I2C	Synchronous half duplex	2 + GND	~1 m (400 pF bus limit)	100 k / 400 k 1 M / 3.4 M	Multi-master, 7-bit addressed	EEPROM, RTC, slow sensors, port expanders. Many devices on two wires.
SPI	Synchronous full duplex	4 + 1 per slave	~30 cm	f(PCLK) / 2 up to 45 Mbit/s	Single master, one CS per slave	SD cards, TFT displays, high-rate IMUs, SPI flash, Ethernet and CAN controller chips.
I2S	Synchronous audio	3 to 4	~30 cm	Depends on sample rate	Point to point	Digital audio codecs, DACs, MEMS microphones. A specialised SPI variant.
1-Wire	Asynchronous half duplex	1 + GND	~100 m	15.4 k 125 k overdrive	Multi-drop, 64-bit ROM ID	DS18B20 temperature sensors, ID chips. Can draw power from the data line itself.
QSPI	Synchronous full duplex	6	~10 cm	~90 Mbit/s per data line	Point to point	External NOR flash, execute-in-place firmware. Four data lines instead of one.
SDIO	Synchronous parallel	6	~10 cm	Up to 208 Mbit/s (UHS-I)	Point to point	SD and microSD cards, eMMC, some Wi-Fi modules. Four-bit data bus.
SMBus	Synchronous half duplex	2 + GND	~1 m	100 k	Multi-master, addressed	Battery packs, power management, PC motherboard sensors. A stricter I2C subset.

Dr. Raghudathesh G P

None of these leave the board. The moment a cable is involved, look at the next slide.

Board-to-Board and Industrial Protocols

Cables, electrical noise, and the differential signalling that survives it

Protocol	Type	Wires	Max distance	Max speed	Topology	Typical usage
UART (TTL)	Asynchronous full duplex	2 + GND	~1 m	Up to ~11 Mbit/s on the F446	Point to point	Board-to-board debug links, GPS modules, Bluetooth and GSM modems. Simplest of all.
RS-232	Asynchronous full duplex	3 to 9	15 m (50 ft)	20 kbit/s spec limit	Point to point	Legacy PC serial ports, industrial instruments, console access. ±12 V single-ended.
RS-422	Asynchronous full duplex	4 + GND	1200 m (4000 ft)	10 Mbit/s	1 driver, up to 10 receivers	Long-run point-to-point telemetry, motor encoders. Differential pairs.
RS-485	Asynchronous half duplex	2 + GND	1200 m (4000 ft)	10 Mbit/s	Multi-drop, 32+ nodes	Factory automation, Modbus, building control, DMX lighting. The industrial workhorse.
CAN 2.0B	Asynchronous half duplex	2 + GND	40 m at 1 Mbit/s	1 Mbit/s	Multi-master, message priority	Automotive body and powertrain, industrial machinery. Arbitration without a master.
CAN FD	Asynchronous half duplex	2 + GND	40 m	Up to 8 Mbit/s in data phase	Multi-master	Modern vehicle networks needing larger payloads than classic CAN allows.
LIN	Asynchronous half duplex	1 + GND	40 m	20 kbit/s	Single master, up to 16 slaves	Car door mirrors, seat motors, window switches. A cheap companion to CAN.

Dr. Raghudathesh G P

Differential pairs are what buy distance: RS-485 and CAN reject noise that would destroy RS-232.

Host and Network Protocols

Connecting to a PC or a network, with a full protocol stack behind them

Protocol	Type	Wires	Max distance	Max speed	Topology	Typical usage
`USB 2.0 FS`	Half duplex differential	4	5 m per segment	12 Mbit/s	Tiered star, host polls	Most embedded USB devices: HID, CDC virtual COM ports, small storage.
`USB 2.0 HS`	Half duplex differential	4	5 m per segment	480 Mbit/s	Tiered star	Cameras, external drives, audio interfaces. Needs an external PHY on the F446.
`USB 3.x`	Dual simplex differential	9	3 m typical	5 to 20 Gbit/s	Tiered star	Mass storage, video capture. Beyond the reach of a Cortex-M4.
`Ethernet 10/100`	Serial differential	4 or 8	100 m per segment	10 / 100 Mbit/s	Switched star	Industrial networking, IoT gateways, PLCs. Not available on the F446.
`Gigabit Ethernet`	Serial differential	8	100 m	1 Gbit/s	Switched star	Building and campus networks, machine vision cameras.
`10G Ethernet`	Serial differential	8 / fibre	100 m copper	10 Gbit/s	Switched star	Data centre backbones, high-bandwidth network cores.
`SWD`	Synchronous half duplex	2 + GND	~20 cm	Up to 4 MHz typical	Point to point, daisy-chainable	Debug and flash programming. This is what your ST-LINK uses.

Dr. Raghudathesh G P

Why published distance figures disagree

Quoted maxima assume a specific cable, data rate and termination, and distance trades against speed. Design against the standard, not a comparison table.

Choosing a Protocol — A Practical Guide

It stays on one board, and it is fast	→	**SPI**	Displays, SD cards, high-rate sensors, external flash.
It stays on one board, and there are several slow devices	→	**I2C**	EEPROM, RTC, temperature sensors. Two wires serve them all.
It leaves the board and goes to a PC	→	**UART, then USB**	A virtual COM port over the ST-LINK is the cheapest link you will ever build.
It runs tens of metres through a noisy plant	→	**RS-485**	Differential, multi-drop, and the physical layer under Modbus.
Several nodes must talk without a master	→	**CAN**	Priority arbitration built into the physical layer. Automotive standard.
It needs a network address and a switch	→	**Ethernet**	Requires a MAC and PHY the STM32F446 does not have.

Dr. Raghudathesh G P

If it is fast or it is a display, use SPI. If it is slow and there are several, use I2C. If it leaves for a PC, use UART.

What the STM32F446RET6 Actually Provides

Which of these protocols you can build without adding a transceiver chip

Built in, no extra parts

I2C x4, SPI x4, I2S x3, USART x4, UART x2, SDIO, QuadSPI, USB OTG FS

Built in, needs a transceiver

CAN x2 (needs a CAN transceiver), RS-485 and RS-232 (a USART plus a line driver), LIN (a USART in LIN mode)

Built in, needs an external PHY

USB OTG HS

Not available on this chip

Ethernet MAC, 1-Wire hardware (can be bit-banged on a GPIO), CAN FD

A transceiver is not a protocol

CAN, RS-485 and RS-232 all start life as a digital signal from the MCU. The transceiver chip only converts voltage levels and adds differential drive.

What you have already built

GPIO, EXTI, USART and ADC drivers. SPI and I2C reuse exactly the same driver pattern — register structure, config struct, handle, blocking and interrupt APIs.

Dr. Raghudathesh G P

The Three You Will Use Most

UART

2 wires, no clock

Both ends preset to the same baud rate. A mismatch corrupts data with no warning. Full duplex, point to point, no addressing, no acknowledgement.

Typical use

PC terminals, GPS, modems, board-to-board debug

I2C

2 wires, shared clock

Every device has a 7-bit address. Open-drain lines need external pull-ups. Half duplex, multi-master, and every byte is acknowledged.

Typical use

EEPROM, RTC, slow sensors, port expanders

SPI

4 wires, shared clock

No addressing — a separate chip select per slave. Full duplex, so a byte goes out for every byte that comes in. Fast, but no acknowledgement at all.

Typical use

Displays, SD cards, flash, fast sensors

Dr. Raghudathesh G P

The difference that catches everyone

I2C acknowledges every byte, so a missing device is reported immediately as a NACK. SPI and UART acknowledge nothing — if the partner is absent you read 0xFF or 0x00 forever with no error anywhere. That is why SPI devices provide a WHO_AM_I register returning a known constant.